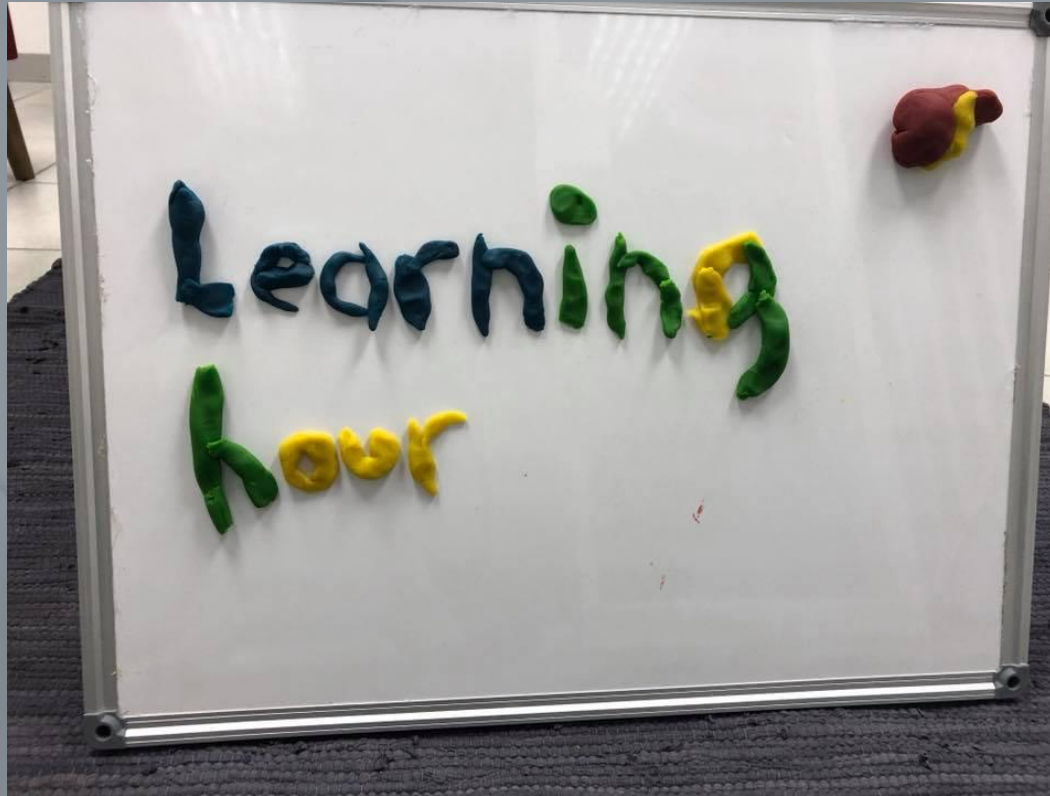




Jan Leßner, S&N Invent GmbH

Learning Hour Agenda

1. Why
2. Design Principle „Integration Operation Segregation“
3. Design Principle „Single Responsibility“
4. Example Kata „Social Time“
5. Playing around

Why?

- Problematic inherited Code from Raiffeisen
 - Lack of clean code design
 - Can be addressed by well-proven design principles
 - Probably known but not yet consequently applied
 - GI Joe Fallacy
 - Learning Hour makes everybody really **do** it
 - On a „beginners' hill“
 - In the afternoon: transfer it to the real world
-

Lets have a look

Class: **ExchangeRateAccess**

Method: **getExchangeRateNormalMode**

size doesn't matter ;-)

<https://clean-code-developer.com/>
<https://clean-code-developer.de/>

Roter Grad

Inhaltsverzeichnis
1. Prinzipien
1.1. <u>Don't Repeat Yourself (DRY)</u>
1.2. <u>Keep it simple, stupid (KISS)</u>
1.3. <u>Beware of Premature Optimization</u>
1.4. <u>Favour Composition over Inheritance (FCol)</u>
1.5. <u>Integration Operation Segregation Principle (IOSP)</u>
2. <u>Praktiken</u>
2.1. <u>Boy Scout Rule</u>
2.2. <u>Root Cause Analysis</u>
2.3. <u>Version Control System</u>
2.4. <u>Simple Refactorings</u>
2.5. <u>Daily Reflection</u>

Orangener Grad

Inhaltsverzeichnis
1. Prinzipien
1.1. <u>Single Level of Abstraction</u>
1.2. <u>Single Responsibility Principle (SRP)</u>
1.3. <u>Separation of Concerns (SoC)</u>
1.4. <u>Source Code Conventions</u>
2. <u>Praktiken</u>
2.1. <u>Issue Tracking</u>
2.2. <u>Automated Integrationtests</u>
2.3. <u>Read, Read, Read</u>
2.4. <u>Reviews</u>

Integration Operation Segregation

... as it is explained on clean-code-developer.com

If a method contains both logic and calls to other methods, the total behavior will be no longer clear. The instructions are blurred over a possibly very deep hierarchy. Furthermore this kind of method has a tendency to grow unlimited.

Integration Operation Segregation calls for a clear separation:

- Either a method contains exclusively logic, meaning transformations, control structures or API invocations. Then it's called an Operation.
- Or a method does not contain any logic but exclusively calls other methods within its code basis. Then it's called Integration.

Any developer willing can apply this principle off-hand. Compliance is trivial to verify as integration and operation differ notably in form.

Single Level of Abstraction

(Big brother of „Integration Operation Segregation Principle“)

Lines of code may have different levels of abstraction. Assigning a value to a variable is on a lower level than let's say a method call. This is because beyond the method call can be far more logic than a variable assignment. Even method calls may be on different levels of abstraction. Calling a framework's method is on a different level than a method call within your codebase.

Within one method you should use only one level of abstraction. Otherwise the reader will find it hard to differentiate essentials from details. So if you need to fiddle with bits, don't combine it with method calls.

A helpful analogy is a newspaper article: First comes is the most important thing – the heading. From this you should roughly know what the article is about. The first sentence of the article describes it on a high level of abstraction. The further you get in the article the more details emerge. We can structure our code in a similar way. Class name is the heading. Next are public methods on a high level of abstraction. These may call lower level methods until we are finally at “fiddle-with-bits” level. With this structure at hand I can easily decide which detail level I want to see when looking at a class. If you are only roughly interested in how a class works, you just need to inspect the public methods which reflect functionality on a high level of abstraction. If you want to know further details, you will drill down to private methods.

Single Responsibility

Single Responsibility Principle (SRP) is one of the SOLID principles stating that a class shall have only one responsibility.

Beyond the Single Responsibility Principle stands the idea that changing or enhancing functionality should affect only a few classes. The more classes need to be changed the higher is the risk that problems arise in areas which basically have nothing to do with the enhancement. A SRP violation leads to coupling and increased complexity. It becomes harder to understand the code.

Example Kata „Social Time“

- Simple code kata
 - Display age of a social media post in a reader-friendly way
 - Requirements revealed stepwise
 - Allows to produce spaghetti code in a time lapse
 - Only a few lines of code
-